

Contents

1	DP	1
1.1	LCS	1
1.2	LIS $O(n^2)$	1
1.3	LIS $O(n \log n)$	1
1.4	LIS $O(n \log n)$	1
2	Prime	2
2.1	質數篩 CPP	2
2.2	質數篩 PY	2
2.3	單一質數	2
2.4	egcd CPP	2
2.5	egcd PY	2
2.6	SPF and maximum factor	2
2.7	Sieve of Eratosthenes	2
3	Graph	3
3.1	Floyd Warshall	3
3.2	Bellman Ford	3
3.3	SPFA	3
3.4	Dijkstra	3
3.5	SecondSP	4
3.6	Kruskal's Algorithm	4
3.7	SecondMST	5
3.8	Prim's Algorithm	5
3.9	LCA	6
3.10	Topological Sort	6
3.11	Articulation Point	6
3.12	Kosaraju Algorithm	7
3.13	Tarjan SCC	7
3.14	Tree Diameter	8
4	RMQ	8
4.1	Segment Tree V1	8
4.2	Segment Tree V2	8
4.3	BIT	9
4.4	Sparse Table	10
5	Uncategorized	10
5.1	快速幂	10
5.2	矩陣快速幂	10
5.3	快速計算費氏數列	10
6	常用小扣	11
6.1	01 背包問題	11
6.2	Topological	11
6.3	常用	11
7	Basic	11
7.1	GCD	11
7.2	LCM	11
7.3	Leap Year	11
8	Trick	11
8.1	求根號和解決 pow 精度問題	11
8.2	查找和二分搜	12
8.3	$a^b \bmod p$	12
8.4	$n! \bmod p$	12
8.5	從 1 ~ n 選 M 個數字	12
8.6	二項式係數 Binomial Coefficient (也可以用在 C 幾取幾)	12
8.7	LICS	12
8.8	m-ary to 10-ary	12
9	Math	12
9.1	ModInverse	12
9.2	CRT	12
9.3	LinearCongruence	13
10	Temporary	13
10.1	Disjoint Set v1	13
10.2	Disjoint Set v2	13
10.3	Tree DP	13

1 DP

1.1 LCS

```

1 #include <bits/stdc++.h>
2 #define IOS
   ios_base::sync_with_stdio(false);cin.tie(0);cout.tie(0);
3 using namespace std;
4 string s1, s2;
5 int dp[505][505];
6 int main(){

```

```

7   IOS
8   cin >> s1 >> s2;
9   memset(dp, 0, sizeof(dp));
10  int l1 = s1.size(), l2 = s2.size();
11  for(int i = 1; i <= l1; i++){
12      for(int j = 1; j <= l2; j++){
13          if(s1[i - 1] == s2[j - 1]) dp[i][j] =
              dp[i - 1][j - 1] + 1;
14          else dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
15      }
16  }
17  cout << dp[l1][l2] << '\n';
18
19  return 0;
20 }

```

1.2 LIS $O(n^2)$

```

1 #include <bits/stdc++.h>
2 #define IOS
   ios_base::sync_with_stdio(false);cin.tie(0);cout.tie(0);
3 using namespace std;
4 typedef long long ll;
5 int main(){
6     IOS
7     int arr[100];
8     int n;
9     cin >> n;
10    for(int i = 0; i < n; i++) cin >> arr[i];
11    int dp[100];
12    for(int i = 0; i < n; i++) dp[i] = 1;
13    for(int i = 0; i < n; i++){
14        for(int j = 0; j < i; j++){
15            if(arr[i] > arr[j])
16                dp[i] = max(dp[j] + 1, dp[i]);
17        }
18    }
19    int ans = 1;
20    for(int i = 0; i < n; i++) ans = max(ans, dp[i]);
21    cout << ans << '\n';
22
23    return 0;
24 }

```

1.3 LIS $O(n \log n)$

```

1 class Solution {
2 public:
3     int lengthOfLIS(vector<int>& nums) {
4         vector<int> v;
5         int n = nums.size();
6         for(int i = 0; i < n; i++){
7             int p = lower_bound(v.begin(), v.end(),
                nums[i]) - v.begin();
8             if(p == v.size()) v.push_back(nums[i]);
9             else v[p] = nums[i];
10        }
11        return v.size();
12    }
13 };

```

1.4 LIS $O(n \log n)$

```

1 for(int i=0;i<num.size();i++){
2     if(lis.empty()||lis.back()<num[i]){
3         lis.push_back(num[i]);
4         dp[i]=lis.size();
5     }
6     else{
7         auto iter=lower_bound(all(lis),num[i]);

```

```

8         dp[i]=iter-lis.begin()+1;
9         *iter=num[i];
10    }
11 }
12 int length=lis.size();
13 for(int i=num.size()-1;i>=0;i--){
14     if(dp[i]==length){
15         ans.push_back(num[i]);
16         length--;
17     }
18 }

```

2 Prime

2.1 質數篩 CPP

```

1 bitset<MAXN> prime_bool;
2 vector<ll> prime;
3 void find_prime(){
4     prime_bool.set();
5     for(int i=2;i<MAXN;i++){
6         if(prime_bool[i]){
7             prime.push_back(i);
8         }
9         for(auto j:prime){
10            if(j*i>=MAXN)
11                break;
12            prime_bool[j*i]=0;
13            if(i%j==0)
14                break;
15        }
16    }
17 }

```

2.2 質數篩 PY

```

1 is_prime = n * [1]
2 is_prime[0] = is_prime[1] = 0
3
4 for i in range(2, n):
5     if is_prime[i]:
6         for j in range(2, n):
7             if i * j >= n:
8                 break
9             is_prime[i * j] = 0

```

2.3 單一質數

```

1 bool prime(int n){
2     if(n < 2) return false;
3     if(n <= 3) return true;
4     if(!(n % 2) || !(n % 3)) return false;
5     for(int i = 5; i * i <= n; i += 6)
6         if(!(n % i) || !(n % (i + 2))) return false;
7     return true;
8 }

```

2.4 egcd CPP

```

1 int exgcd(int a,int b,int &x,int &y){
2     if(b==0){
3         x=1,y=0;
4         return a;
5     }
6     int gcd=exgcd(b,a%b,y,x);
7     y-=a/b*x;
8     return gcd;
9 }

```

2.5 egcd PY

```

1 def egcd(a: int, b: int) -> Tuple[int, int, int]:
2     """return (g, x, y) such that a*x + b*y = g =
3         gcd(a, b)"""
4     if a == 0:
5         return (b, 0, 1)
6     else:
7         b_div_a, b_mod_a = divmod(b, a)
8         g, x, y = egcd(b_mod_a, a)
9         return (g, y - b_div_a * x, x)

```

2.6 SPF and maximum factor

```

1 #include <iostream>
2 #include <cstring>
3 #define IOS
4     ios_base::sync_with_stdio(false);cin.tie(0);cout.tie(0);
5 using namespace std;
6 int spf[1000005], ans[1000005];
7 void sieve(){
8     memset(spf, 0, sizeof(spf));
9     for (int i = 0; i <= 1000000; i++) ans[i] = 1;
10    spf[1] = 1;
11    for(long long i = 2; i <= 1000000; i++){
12        if(!spf[i]){
13            spf[i] = i;
14            for(long long j = i * i; j <=
15                1000000; j+=i){
16                if(!spf[j]){
17                    spf[j] = i;
18                }
19            }
20        }
21        for(long long i = 2; i <= 1000000; i++){
22            int tmp = i;
23            while(tmp != 1){
24                int cnt = 0, f = spf[tmp];
25                while(tmp % f == 0){
26                    tmp /= spf[tmp];
27                    cnt++;
28                }
29                ans[i] *= (cnt + 1);
30            }
31        }
32        int maxn = 2, maxans = ans[2];
33        for(long long i = 2; i <= 1000000; i++){
34            if(ans[i] >= maxans){
35                maxans = ans[i];
36                maxn = i;
37            }
38            ans[i] = maxn;
39        }
40    }
41    int main(){
42        IOS
43        int t;
44        sieve();
45        cin >> t;
46        while(t--){
47            int n;
48            cin >> n;
49            cout << ans[n] << '\n';
50        }
51    }
52    return 0;

```

2.7 Sieve of Eratosthenes

```

1 //n為會跑到的最大值
2 bool isprime[1000005]; //紀錄每個數字是否是質數
3 vector<int> prime; // 儲存範圍內所有的質數
4 isprime[1]=1; // 1 代表此數字不是質數，否則為 0
5 //先將所有大於1數字設為質數(0)
6
7 for(int i=2;i<=n;i++){
8     if(!isprime[i]){ //如果為質數
9         prime.push_back(i);
10        for(long long j=2;i*j<=n;j++){
11            //記得容易會爆int的話要設long long
12            //所有質數的倍數設成非質數
13            isprime[i*j] = 1;
14        }
15    }

```

3 Graph

3.1 Floyd Marshall

```

1 int n, rd, l, r, v;
2 cin >> n >> rd;
3 vector<vector<int>> dp(n + 1, vector<int>(n + 1,
4     1e9));
5 for(int i = 0; i < rd; i++){
6     cin >> l >> r >> v;
7     dp[l][r] = dp[r][l] = v;
8     //每條路皆雙向
9 }
10 //以下即 Floyd-Marshall
11 for(int k = 1; k <= n; k++){
12     for(int i = 1; i <= n; i++){
13         for(int j = 1; j <= n; j++){
14             dp[i][j] = min(dp[i][k] + dp[k][j],
15                 dp[i][j]);
16             //窮舉所有鬆弛可能
17         }
18     }
19 }
20 cin >> l >> r;
21 cout << dp[l][r];

```

3.2 Bellman Ford

```

1 #include <bits/stdc++.h>
2 #define IOS
3 ios_base::sync_with_stdio(false);cin.tie(0);cout.tie(0)
4 #define INF 0x3f3f3f3f
5 using namespace std;
6 typedef long long ll;
7 struct Edge{
8     int x, y, t;
9 };
10 int dis[1005];
11 int main(){
12     IOS
13     int c;
14     cin >> c;
15     while(c--){
16         vector<Edge> edge;
17         int n, m;
18         cin >> n >> m;
19         for(int i = 0; i <= n; i++) dis[i] = INF;
20         dis[0] = 0;
21         for(int i = 0; i < m; i++){
22             int x, y, t;
23             cin >> x >> y >> t;
24             edge.push_back({x, y, t});
25         }

```

```

25     for(int i = 0; i < n - 1; i++){
26         for(int j = 0; j < m; j++){
27             if(dis[edge[j].x] + edge[j].t <
28                 dis[edge[j].y]){
29                 dis[edge[j].y] = dis[edge[j].x] + edge[j].t;
30             }
31         }
32         bool judge = true;
33         for(auto e : edge){
34             if(dis[e.x] + e.t < dis[e.y]){
35                 judge = false;
36                 break;
37             }
38         }
39         cout << (judge ? "not possible" : "possible") <<
40             '\n';
41     }
42     return 0;
43 }

```

3.3 SPFA

```

1 #define mem(x) memset(x, 0, sizeof(x))
2 struct road{
3     int r, val;
4 };
5 int main(){
6     int n, e, l, r, v;
7     cin >> n >> e;
8     vector<int> dp(n + 1, 1e9);
9     vector<road> rd[n + 1];
10    for(int i = 0; i < e; i++){
11        cin >> l >> r >> v;
12        rd[l].push_back({r, v});
13        rd[r].push_back({l, v});
14    }
15    cin >> l >> r;
16    dp[l] = 0;
17    queue<int> que;
18    que.push(l);
19    bool check[n + 1]; mem(check);
20    int cnt[n + 1]; mem(cnt);
21    while(!que.empty()){
22        int tmp = que.front(); que.pop();
23        check[tmp] = 0, cnt[tmp]++;
24        if(cnt[tmp] >= n) {cout << "neg cycle\n"; break;}
25        for(auto & i : rd[tmp]){
26            if(dp[i.r] > dp[tmp] + i.val){
27                dp[i.r] = dp[tmp] + i.val;
28                if(!check[i.r]) check[i.r] = 1, que.push(i.r);
29            }
30        }
31    }
32    for(auto & i : dp) cout << i << ' ';
33    return 0;
34 }

```

3.4 Dijkstra

```

1 #include <iostream>
2 #include <algorithm>
3 #include <vector>
4 #include <queue>
5 #define IOS
6 ios_base::sync_with_stdio(false);cin.tie(0);cout.tie(0);
7 #define INF 2147483647
8 using namespace std;
9 int n, m;
10 vector<pair<int, int>> adj[100005];
11 bool visited[100005] = {false};
12 priority_queue<pair<int, int>> pq;

```

```

12 int dis[100005], parent[100005];
13 void solve(){ // Dijkstra
14     dis[0] = 0;
15     for(int i = 1; i < n; i++) dis[i] = INF;
16     pq.push(make_pair(0, 0));
17     while(!pq.empty()){
18         auto node = pq.top();
19         pq.pop();
20         int v = node.second; // parent
21         if(visited[v]) continue;
22         visited[v] = true;
23         for(auto i : adj[v]){
24             int vertex = i.first, weight = i.second;
25             if(visited[vertex]) continue;
26             if(dis[v] + weight < dis[vertex]){
27                 dis[vertex] = dis[v] + weight;
28                 parent[vertex] = v;
29                 pq.push(make_pair(-dis[vertex],
30                             vertex));
31             }
32         }
33         int maxd = -1, cnt = 0;
34         for(int i = 0; i < n; i++){
35             if(dis[i] < INF){
36                 if(dis[i] > maxd) maxd = dis[i];
37             }
38             else cnt++;
39         }
40         cout << maxd << '\n' << cnt << '\n';
41     }
42     int main(){
43         IOS
44         cin >> n >> m;
45         for(int i = 0; i < m; i++){
46             int u, v, w;
47             cin >> u >> v >> w;
48             adj[u].push_back(make_pair(v, w));
49             adj[v].push_back(make_pair(u, w));
50         }
51         solve();
52
53         return 0;
54 }

```

3.5 SecondSP

```

1 #include <bits/stdc++.h>
2 #define IOS
3 #define INF 0x3f3f3f3f
4 using namespace std;
5
6 typedef pair<int, int> pii;
7 typedef long long ll;
8
9 const int MAXN = 1005;
10
11 vector<pii> adj[MAXN]; // adj[u] stores pairs {v,
12 // weight}
13 int dist[MAXN], sec_dist[MAXN]; // shortest and
14 // second shortest distances
15 void dijkstra(int s, int n){
16     // Min-heap to store {distance, node}
17     priority_queue<pii, vector<pii>, greater<pii>> pq;
18     fill(dist, dist + n + 1, INF);
19     fill(sec_dist, sec_dist + n + 1, INF);
20
21     dist[s] = 0;
22     pq.push({0, s});
23
24     while(!pq.empty()){
25         int d = pq.top().first;
26         int u = pq.top().second;
27         pq.pop();

```

```

28         // If we found a path longer than the second
29         // shortest, skip it
30         if (sec_dist[u] < d) continue;
31
32         for (auto &edge : adj[u]){
33             int v = edge.first;
34             int w = edge.second;
35
36             int new_dist = d + w;
37
38             // If this gives a new shortest path to v
39             if(new_dist < dist[v]){
40                 sec_dist[v] = dist[v];
41                 dist[v] = new_dist;
42                 pq.push({dist[v], v});
43                 pq.push({sec_dist[v], v});
44             }
45             // If this gives a new second shortest
46             // path to v
47             else if(new_dist > dist[v] && new_dist <
48                 sec_dist[v]){
49                 sec_dist[v] = new_dist;
50                 pq.push({sec_dist[v], v});
51             }
52         }
53     }
54 }
55
56 int main() {
57     IOS
58     int t;
59     cin >> t;
60     while(t--){
61         int n, m, s, d;
62         cin >> n >> m >> s >> d;
63
64         // Reset adjacency list
65         for(int i = 1; i <= n; i++) adj[i].clear();
66
67         for(int i = 0; i < m; i++){
68             int u, v, w;
69             cin >> u >> v >> w;
70             adj[u].push_back({v, w});
71             adj[v].push_back({u, w}); // If the graph
72             // is undirected
73         }
74
75         dijkstra(s, n);
76
77         if(sec_dist[d] == INF) cout << -1 << '\n'; //
78         // No second shortest path
79         else cout << sec_dist[d] << '\n';
80     }
81
82     return 0;
83 }

```

3.6 Kruskal's Algorithm

```

1 #include <bits/stdc++.h>
2 #define IOS
3 ios_base::sync_with_stdio(false); cin.tie(0); cout.tie(0);
4 using namespace std;
5 int parent[100005];
6 struct Edge{
7     int u, v, w;
8     bool operator < (Edge &b){
9         return w < b.w;
10     }
11 };
12 int query(int a){
13     if(parent[a] == -1) return a;
14     return parent[a] = query(parent[a]);
15 }
16 bool merge(int a, int b){

```

```

16     int r1 = query(a);
17     int r2 = query(b);
18     if(r1 == r2) return false;
19     if(parent[r1] < parent[r2]) parent[r2] = r1;
20     else parent[r1] = r2;
21     return true;
22 }
23 int main(){
24     IOS
25     int n, m;
26     memset(parent, -1, sizeof(parent));
27     cin >> n >> m;
28     vector<Edge> adj;
29     for(int i = 0; i < m; i++){
30         int u, v, w;
31         cin >> u >> v >> w;
32         adj.push_back({u, v, w});
33     }
34     sort(adj.begin(), adj.end());
35     // for(int i = 0; i < m; i++) cout << adj[i].w << '
36         ';
37     int cost = 0, n_edge = 0;
38     for(Edge e : adj){
39         if(merge(e.u, e.v)){
40             cost += e.w;
41             n_edge++;
42         }
43     }
44     if(n_edge == n - 1) cout << cost << '\n';
45     else cout << -1 << '\n';
46     return 0;
47 }

```

```

40     sort(edge.begin(), edge.end(), [&](Edge &u,
41         Edge &v){return u.c < v.c;});
42     int cost1 = 0, cnt = 0;
43     vector<int> mst;
44     for(int i = 0; i < edge.size(); i++){
45         if(merge(edge[i].a, edge[i].b)){
46             cost1 += edge[i].c;
47             mst.push_back(i);
48             if(++cnt == n - 1) break;
49         }
50     }
51     int cost2 = INF;
52     for(int i = 0; i < mst.size(); i++){
53         cnt = 0;
54         int res = 0;
55         for(int i = 1; i <= n; i++) anc[i] = i;
56         for(int j = 0; j < edge.size(); j++){
57             if(mst[i] == j) continue;
58             if(merge(edge[j].a, edge[j].b)){
59                 res += edge[j].c;
60                 if(++cnt == n - 1){
61                     cost2 = min(cost2, res);
62                     break;
63                 }
64             }
65         }
66     }
67     cout << cost1 << ' ' << cost2 << '\n';
68 }
69 return 0;
70 }
71 }

```

3.7 SecondMST

```

1 #include <bits/stdc++.h>
2 #define IOS
3     ios_base::sync_with_stdio(false); cin.tie(0); cout.tie(0);
4 #define INF 0x3f3f3f3f
5 using namespace std;
6 typedef long long ll;
7 int anc[105], sz[105];
8 struct Edge{
9     int a, b, c;
10 };
11 int Find(int a){
12     return (anc[a] == a ? a : anc[a] = Find(anc[a]));
13 }
14 bool merge(int a, int b){
15     a = Find(a);
16     b = Find(b);
17     if(a == b) return false;
18     if(sz[a] < sz[b]) swap(a, b);
19     anc[b] = a;
20     sz[a] += sz[b];
21     return true;
22 }
23 int main(){
24     IOS
25     int t;
26     cin >> t;
27     while(t--){
28         int n, m;
29         vector<Edge> edge;
30         cin >> n >> m;
31         vector<pair<int, int>> vec[105];
32         for(int i = 0; i < m; i++){
33             int a, b, c;
34             cin >> a >> b >> c;
35             edge.push_back({a, b, c});
36         }
37         for(int i = 1; i <= n; i++){
38             sz[i] = 0;
39             anc[i] = i;

```

3.8 Prim's Algorithm

```

1 #include <iostream>
2 #include <queue>
3 #include <algorithm>
4 #include <cstring>
5 #define IOS
6     ios_base::sync_with_stdio(false); cin.tie(0); cout.tie(0);
7 using namespace std;
8 int n, m, dis[10005], parent[10005];
9 bool visited[10005] = {false};
10 vector<pair<int, int>> adj[10005];
11 int main(){
12     IOS
13     // freopen("input.in", "r", stdin);
14     cin >> n >> m;
15     memset(dis, 0x3f3f3f3f, sizeof(dis));
16     memset(parent, -1, sizeof(parent));
17     for(int i = 0; i < m; i++){
18         int u, v, w;
19         cin >> u >> v >> w;
20         adj[u].push_back({v, w});
21         adj[v].push_back({u, w});
22     }
23     int start = 0;
24     dis[start] = 0;
25     priority_queue<pair<int, int>, vector<pair<int, int>, greater<pair<int, int>>> pq;
26     pq.push({dis[start], start});
27     while(!pq.empty()){
28         pair<int, int> cur = pq.top();
29         pq.pop();
30         if(visited[cur.second]) continue;
31         visited[cur.second] = true;
32         for(auto i : adj[cur.second]){
33             if(visited[i.first]) continue;
34             if(dis[i.first] > i.second){
35                 dis[i.first] = i.second;
36                 parent[i.first] = cur.second;
37                 pq.push({dis[i.first], i.first});
38             }

```

```

39     }
40     int cost = 0, err = 0;
41     for(int i = 0; i < n; i++){
42         if(dis[i] < 0x3f3f3f3f) cost += dis[i];
43         else err++;
44     }
45     cout << (err ? -1 : cost) << "\n";
46     // for(int i = 0; i < n; i++) cout << dis[i] << ' ';
47
48     return 0;
49 }

```

3.9 LCA

```

1 #include <bits/stdc++.h>
2 #define IOS
3     ios_base::sync_with_stdio(false); cin.tie(0); cout.tie(0);
4 #define INF 0x3f3f3f3f
5 using namespace std;
6 typedef long long ll;
7 const int N = 2e5 + 5;
8 int n, q;
9 vector<int> vec[N];
10 int p[20][N], in[N], out[N];
11 bool valid(int a, int b){
12     return (in[a] <= in[b] && out[b] <= out[a]);
13 }
14 void dfs(int cur, int par){
15     static int t = 0;
16     p[0][cur] = par;
17     in[cur] = t++;
18     for(auto e : vec[cur]){
19         dfs(e, cur);
20     }
21     out[cur] = t++;
22 }
23 int lca(int a, int b){
24     if(valid(a, b)) return a;
25     for(int i = 19; i >= 0; i--){
26         if(!valid(p[i][a], b)) a = p[i][a];
27     }
28     return p[0][a];
29 }
30 int main(){
31     IOS
32     cin >> n >> q;
33     for(int i = 2; i <= n; i++){
34         int e;
35         cin >> e;
36         vec[e].push_back(i);
37     }
38     dfs(1, 1);
39     for(int i = 1; i < 20; i++){
40         for(int j = 1; j <= n; j++){
41             p[i][j] = p[i - 1][p[i - 1][j]];
42         }
43     }
44     while(q--){
45         int u, v;
46         cin >> u >> v;
47         cout << lca(u, v) << '\n';
48     }
49     return 0;
50 }

```

3.10 Topological Sort

```

1 #include <bits/stdc++.h>
2 #define IOS
3     ios_base::sync_with_stdio(false); cin.tie(0); cout.tie(0);
4 using namespace std;
5 typedef long long ll;

```

```

5 vector<int> vec[200005];
6 int ind[100005];
7 int main(){
8     IOS
9     int n, m;
10    cin >> n >> m;
11    memset(ind, 0, sizeof(ind));
12    for(int i = 0; i < m; i++){
13        int a, b;
14        cin >> a >> b;
15        ind[b]++;
16        vec[a].push_back(b);
17    }
18    queue<int> q;
19    for(int i = 1; i <= n; i++){
20        if(ind[i] == 0) q.push(i);
21    }
22    vector<int> top;
23    while(!q.empty()){
24        int cur = q.front();
25        q.pop();
26        top.push_back(cur);
27        for(auto e : vec[cur]){
28            ind[e]--;
29            if(ind[e] == 0){
30                q.push(e);
31            }
32        }
33    }
34    if(top.size() == n){
35        for(auto i : top) cout << i << ' ';
36        cout << '\n';
37    }
38    else cout << "IMPOSSIBLE" << '\n';
39
40    return 0;
41 }

```

3.11 Articulation Point

```

1 #include <iostream>
2 #include <vector>
3 #include <string>
4 #include <sstream>
5
6 using namespace std;
7
8 vector< vector<int> > edge;
9 vector<int> dfn, low;
10 int result, dep;
11
12 void dfs(int cur, int par)
13 {
14     bool cut = false;
15     int child = 0;
16     dfn[cur] = low[cur] = ++dep;
17
18     for (auto& i : edge[cur]) {
19
20         if (!dfn[i]) {
21             ++child;
22             dfs(i, cur);
23             low[cur] = min(low[cur], low[i]);
24             if (low[i] >= dfn[cur]) cut = true;
25         }
26         else if (i != par) low[cur] = min(low[cur], dfn[i]);
27     }
28
29     if ((child >= 2 || par != -1) && cut) ++result;
30 }
31
32 int main()
33 {
34     ios::sync_with_stdio(false);
35     cin.tie(nullptr);

```

```

36 cout.tie(nullptr);
37
38 int N;
39 while (cin >> N && N) {
40
41     cin.ignore();
42     result = 0, dep = 0;
43     edge.assign(N + 1, vector<int>());
44     dfn.assign(N + 1, 0);
45     low.assign(N + 1, 0);
46
47     string str;
48     while (getline(cin, str)) {
49         stringstream ss(str);
50         int u, v;
51
52         ss >> u;
53         if (!u) break;
54         while (ss >> v) edge[v].emplace_back(u),
55             edge[u].emplace_back(v);
56     }
57
58     dfs(1, -1);
59
60     cout << result << "\n";
61 }
62
63 return 0;
64 }

```

3.12 Kosaraju Algorithm

```

1 #define MAXN 100005
2
3 stack<int> stk; //stack
4 vector<int> Graph[MAXN]; //graph
5 vector<int> Rev_Graph[MAXN]; //reverse graph
6 bool vis[MAXN]; //is visited
7 int scc[MAXN]; //i's scc
8 int sccid; //scc number
9
10 void dfs1(int now){
11     vis[now]=true;
12     for(auto i:Graph[now]){
13         if(!vis[i]) dfs1(i);
14     }
15     stk.push(now); //push now in stack before return
16 }
17
18 void dfs2(int now){
19     vis[now]=true;
20     scc[now]=sccid; //combine to scc
21     for(auto i:Rev_Graph[now]){
22         if(!vis[i]) dfs2(i);
23     }
24 }
25
26 int main(){
27     int n,m;
28     cin >> n >> m;
29     for(int i=0;i<m;i++){
30         int u,v;
31         cin >> u >> v;
32         Graph[u].push_back(v);
33         Rev_Graph[v].push_back(u);
34     }
35     for(int i=1;i<=n;i++){
36         if(!vis[i]) dfs1(i); //do dfs1 of all elements
37     }
38     memset(vis,0,sizeof(vis)); //reset vis array
39     while(!stk.empty()){ //iterate all elements of
40         //stack
41         int x=stk.top();stk.pop();
42         if(vis[x]) continue; //skip node which has been
43         //combined to another scc

```

```

42     sccid++;
43     dfs2(x);
44 }
45 }

```

3.13 Tarjan SCC

```

1 #include <iostream>
2 #include <vector>
3 #include <map>
4 #include <stack>
5 #include <algorithm>
6
7 using namespace std;
8
9 map<string, vector<string>> G;
10 map<string, int> dfn, low;
11 vector<string> V;
12 int dep;
13
14 void dfs(string u)
15 {
16     low[u] = dfn[u] = ++dep;
17     V.emplace_back(u);
18
19     for (auto& v : G[u])
20         if (!dfn[v]) dfs(v), low[u] = min(low[u],
21             low[v]);
22         else if (count(V.begin(), V.end(), v)) low[u]
23             = min(low[u], dfn[v]);
24
25     if (dfn[u] == low[u])
26     {
27         string temp;
28         while (1)
29         {
30             temp = V[V.size() - 1];
31             V.pop_back();
32             cout << temp;
33             if (temp != u) cout << ", ";
34             else break;
35         }
36         cout << "\n";
37     }
38 }
39
40 int main()
41 {
42     // fast io
43     ios::sync_with_stdio(false);
44     cin.tie(nullptr);
45     cout.tie(nullptr);
46
47     int n, m, cases = 0;
48     while (cin >> n >> m, n && m)
49     {
50         // init
51         G.clear(), dfn.clear(), low.clear(),
52             V.clear();
53         dep = 0;
54
55         // store data
56         while (m--)
57         {
58             string name, to;
59             cin >> name >> to;
60             G[name].emplace_back(to);
61         }
62
63         // find scc
64         if (cases) cout << "\n";
65         cout << "Calling circles for data set " <<
66             ++cases << ":\n";
67         for (auto& [i, j] : G) if (!dfn[i]) dfs(i);
68     }
69 }

```



```
66     return 0;
67 }
```

3.14 Tree Diameter

```
1 #include <cstdio>
2 #include <vector>
3 using namespace std;
4
5 constexpr int N = 100000 + 10;
6
7 int n, c, d[N];
8 vector<int> E[N];
9
10 void dfs(int u, int fa) {
11     for (int v : E[u]) {
12         if (v == fa) continue;
13         d[v] = d[u] + 1;
14         if (d[v] > d[c]) c = v;
15         dfs(v, u);
16     }
17 }
18
19 int main() {
20     scanf("%d", &n);
21     for (int i = 1; i < n; i++) {
22         int u, v;
23         scanf("%d %d", &u, &v);
24         E[u].push_back(v), E[v].push_back(u);
25     }
26     dfs(1, 0);
27     d[c] = 0, dfs(c, 0);
28     printf("%d\n", d[c]);
29     return 0;
30 }
```

4 RMQ

4.1 Segment Tree V1

```
1 #include <bits/stdc++.h>
2 #define IOS
3 #define L(x) (x << 1)
4 #define R(x) ((x << 1) | 1)
5 using namespace std;
6 typedef long long ll;
7 ll seg[500005 << 2], lazy[500005 << 2];
8 int n, q;
9 void init(){
10     memset(seg, 0, sizeof(seg));
11     memset(lazy, 0, sizeof(lazy));
12 }
13 void build(int x, int l, int r){
14     if(l == r){
15         cin >> seg[x];
16         return;
17     }
18     int mid = (l + r) >> 1;
19     build(L(x), l, mid);
20     build(R(x), mid + 1, r);
21     seg[x] = seg[L(x)] + seg[R(x)];
22 }
23 void push(int pos, int size){
24     lazy[L(pos)] += lazy[pos];
25     lazy[R(pos)] += lazy[pos];
26     seg[pos] = seg[pos] + lazy[pos] * size;
27     lazy[pos] = 0;
28 }
29 void modify(int x, int l, int r, int ql, int qr, int val){
30     if(lazy[x]) push(x, (r - l) + 1);
```

```
31     // seg[x] = seg[L(x)] + (mid - l) * lazy[L(x)] +
32     // seg[R(x)] + (r - mid) * lazy[R(x)];
33     seg[x] += val * (qr - ql + 1);
34     if(ql <= l && qr >= r){
35         lazy[x] += val;
36         return;
37     }
38     int mid = (l + r) >> 1;
39     if(qr <= mid) modify(L(x), l, mid, ql, qr, val);
40     else if(ql > mid) modify(R(x), mid + 1, r, ql, qr, val);
41     else{
42         modify(L(x), l, mid, ql, mid, val);
43         modify(R(x), mid + 1, r, mid + 1, qr, val);
44     }
45 }
46 ll query(int x, int l, int r, int ql, int qr){
47     if(ql <= l && qr >= r) return seg[x] + lazy[x] * (r - l);
48     if(lazy[x]) push(x, (r - l) + 1);
49     int mid = (l + r) >> 1;
50     if(qr <= mid) return query(L(x), l, mid, ql, qr);
51     else if(ql > mid) return query(R(x), mid + 1, r, ql, qr);
52     else return query(L(x), l, mid, ql, mid) + query(R(x), mid + 1, r, mid + 1, qr);
53 }
54 int main(){
55     IOS
56     init();
57     cin >> n;
58     build(1, 1, n);
59     cin >> q;
60     while(q--){
61         int v, x, y, k;
62         cin >> v;
63         if(v == 1){
64             cin >> x >> y >> k;
65             modify(1, 1, n, x, y, k);
66         }
67         else{
68             cin >> x >> y;
69             ll ans = query(1, 1, n, x, y);
70             cout << ans << '\n';
71         }
72     }
73     return 0;
74 }
```

4.2 Segment Tree V2

```
1 struct Node{
2     int left; // 左邊邊界
3     int right; // 右邊邊界
4     int value; // 儲存的值
5     int z; // 區間修改用，如果沒有區間修改就不需要
6 }node[4 * N];
7
8 #define Lson(x) (x << 1) // 左子樹
9 #define Rson(x) ((x << 1) + 1) // 右子樹
10
11 void question(){
12     for(int i = 1; i <= 10; i++) num[i] = i * 123 % 5;
13     // num 為題目產生的一段數列
14     // hash 函數，讓 num 的 i 被隨機打亂
15 }
16
17 void build(int left, int right, int x = 1){
18     // left 為題目最大左邊界，right 為題目最大右邊界，圖上最上面的 root 為第一個節點
19     node[x].left = left; // 給 x 節點左右邊界
20     node[x].right = right;
```



```

21 if(left == right){
22     //如果左右邊界節點相同，表示這裡是葉節點
23     node[x].value = num[left] ; //把 num 值給
24     node[x]
25     //這裡的 num 值表示，我們要在 value 要放的值
26     return ; //向前返回
27 }
28 int mid = (left + right) / 2 ; //切半，產生二元樹
29 //debug
30 //cout << mid << '\n' ;
31 //cout << x << ' ' << node[x].left << ' ' <<
32     node[x].right << ' ' << '\n' ;
33 build(left , mid , Lson(x)) ; //將區間改為 [left,
34     mid] 然後帶給左子樹
35 build(mid + 1 , right , Rson(x)) ; //將區間改為
36     [mid+1, right] 然後帶給右子樹
37 node[x].value = min(node[Lson(x)].value ,
38     node[Rson(x)].value ) ;
39 //查詢左右子樹哪個數值最小，並讓左右子樹最小值表示此區
40 }
41 void modify(int position , int value , int x = 1 ){
42     //修改數字
43     if(node[x].left == position && node[x].right ==
44         position ){ //找到葉節點
45         node[x].value = value ; //修改
46         return ; //傳回
47     }
48     int mid = (node[x].left + node[x].right) / 2 ;
49     //切半，向下修改
50     if(position <= mid )
51         //如果要修改的點在左邊，就往左下角追蹤
52         modify(position , value , Lson(x)) ;
53     if(mid < position )
54         //如果要修改的點在右邊，就往右下角追蹤
55         modify(position , value , Rson(x)) ;
56     node[x].value = min(node[Lson(x)].value ,
57         node[Rson(x)].value ) ;
58     //比較左右子樹哪個值比較小，較小值為此節點的 value
59 }
60 void push_down(int x , int add){
61     //將懶人標記往下推，讓下一層子樹進行區間修改
62     int lson = Lson(x) , rson = Rson(x) ;
63     node[lson].z += add ;
64     //給予懶人標記，表示子樹如果要給子樹的子樹區間修改
65     node[rson].z += add ;
66     //數值要是多少，左右子樹都需要做
67     node[lson].v += add ; //更新左右子樹的值
68     node[rson].v += add ;
69 }
70 void update(int a , int b , int cmd , int x = 1){
71     //a, b 為區間修改的 left and right, cmd 為要增加的數值
72     if(a <= node[x].l && b >= node[x].r){
73         //如果節點的 left and right, 跟 a, b
74         //區間是相等，或更小就，只要在這邊修改 cmd，
75         //就可以讓 node[x].v
76         //的值直接變為區間修改後的數值，
77         //之後如果要讓這查詢向子樹進行區間修改，就用
78         //push_down，
79         //我們這邊的懶人標記就會告訴左右子樹要修改的值為多
80         node[x].v += cmd ; //區間修改後的 v
81         node[x].z = cmd ; //區間修改是要增加多少數值
82         return ;
83     }
84     push_down(x) ; //先將之前的區間查詢修改值，往下給子樹以

```

```

75 //假如當前的 node[x].z 原本是 3，如果沒有
76     push_down(x)，那下面的子樹都沒有被 +3，
77     //導致答案不正確。
78     int mid = (node[x].l+node[x].r) / 2 ;
79     //切半，向下修改
80     if(a <= mid) update(a, b, cmd, Lson(x));
81     //如果要修改的點在左邊，就往左下角追蹤
82     if(b > mid) update(a, b, cmd, Rson(x));
83     //如果要修改的點在右邊，就往右下角追蹤
84     node[x].v = node[Lson(x)].v + node[Rson(x)].v;
85     //比較左右子樹哪個值比較小，較小值為此節點的 value
86 }
87 #define INF 0x3f3f3f
88 int query(int left , int right , int x = 1 ){
89     if(node[x].left >= left && node[x].right <= right)
90         return node[x].Min_Value ;
91     //如果我們要查詢的區間比當前節點的區間大，那我們不需再向下查
92     // 例如我們要查詢 [2, 8]，我們只需要查詢 [3,
93     4]，不須查詢 [3, 3]、[4, 4]，
94     // [3, 4] 已經做到最小值查詢
95     push_down(x); //有區間修改時才需要寫
96     int mid = (node[x].left + node[x].right) / 2 ;
97     //切半，向下修改
98     int ans = INF ; //一開始先假設答案為最大值
99     if( left <= mid )
100         //如果切半後，我們要查詢的區間有在左子樹就向下查詢
101         ans = min(ans , query(left , right ,
102             Lson(x))) ; //更新答案，比較誰比較小
103     if(mid < right )
104         //如果切半後，我們要查詢的區間有在右子樹就向下查詢
105         ans = min(ans , query(left , right ,
106             Rson(x))) ; //更新答案，比較誰比較小
107     return ans ; //回傳答案
108 }

```

4.3 BIT

```

1 // BIT
2 #include <bits/stdc++.h>
3 #include <ext/pb_ds/assoc_container.hpp>
4 #include <ext/pb_ds/tree_policy.hpp>
5 // #include <ext/pb_ds/detail/standard_policies.hpp>
6 #define IOS
7 ios_base::sync_with_stdio(false); cin.tie(0); cout.tie(0);
8 #define INF 0x3f3f3f3f
9 #define lowbit(x) x&(-x)
10 using namespace std;
11 using namespace __gnu_pbds;
12 typedef long long ll;
13 const int N = 2e5 + 5;
14 ll bit[N], n, q;
15 ll query(int idx){
16     ll sum = 0;
17     for(int i = idx; i > 0; i -= lowbit(i))
18         sum += bit[i];
19     return sum;
20 }
21 void update(ll val, int idx){
22     for(int i = idx; i <= n; i += lowbit(i))
23         bit[i] += val;
24 }
25 int main(){
26     IOS
27     cin >> n >> q;
28     for(int i = 1; i <= n; i++){ // 1-based
29         ll in;
30         cin >> in;
31         update(in, i);
32     }

```

```

32 while(q--){
33     ll o, a, b;
34     cin >> o >> a >> b;
35     if(o == 1){
36         ll u = query(a) - query(a - 1);
37         update(b - u, a);
38     }
39     else{
40         cout << query(b) - query(a - 1) << '\n';
41     }
42 }
43
44 return 0;
45 }

```

4.4 Sparse Table

```

1 #include <bits/stdc++.h>
2 #define IOS
3 ios_base::sync_with_stdio(false);cin.tie(0);cout.tie(0);
4 #define INF 0x3f3f3f3f
5 using namespace std;
6 typedef long long ll;
7 const int N = 5e5 + 5;
8 int n, m, arr[N], dp[35][N];
9 void sparse_table(int n){
10     for(int i = 1; i <= 31; i++){
11         for(int j = 0; (j + (1LL << (i - 1))) < n; j++){
12             dp[i][j] = max(dp[i - 1][j], dp[i - 1][j
13                 + (1LL << (i - 1))]);
14         }
15     }
16 }
17 int query(int l, int r){
18     int idx = __lg(r - l + 1);
19     return max(dp[idx][l], dp[idx][r - (1LL << idx) +
20         1]);
21 }
22 int main(){
23     IOS
24     cin >> n;
25     for(int i = 0; i < n; i++) cin >> arr[i];
26     cin >> m;
27     for(int i = 0; i < n; i++) dp[0][i] = arr[i];
28     sparse_table(n);
29     while(m--){
30         int l, r;
31         cin >> l >> r;
32         if(l > r) swap(l, r);
33         l--, r--;
34         cout << query(l, r) << '\n';
35     }
36 }

```

5 Uncategorized

5.1 快速幂

```

1 #include <bits/stdc++.h>
2 #define IOS
3 ios_base::sync_with_stdio(false);cin.tie(0);cout.tie(0);
4 using namespace std;
5 typedef long long ll;
6 ll mod = 1000000007;
7 ll fast_pow(int base, int exp){
8     ll res = 1;
9     while(exp > 0){
10         if(exp & 1) res = res * base % mod;
11         base = base * base % mod;
12         exp >>= 1;
13     }
14 }

```

```

12 }
13 return res;
14 }
15 int main(){
16     IOS
17     int base = 3, exp = 15;
18     cout << fast_pow(base, exp) << '\n';
19 }
20 return 0;
21 }

```

5.2 矩陣快速幂

```

1 #include <bits/stdc++.h>
2 #define IOS
3 ios_base::sync_with_stdio(false);cin.tie(0);cout.tie(0);
4 #define INF 0x3f3f3f3f
5 using namespace std;
6 typedef long long ll;
7 ll mod = 1000000007;
8 struct Matrix{
9     ll mat[2][2] = {{0}};
10     Matrix operator * (Matrix &inp){
11         Matrix tmp;
12         for(int i = 0; i < 2; i++){
13             for(int j = 0; j < 2; j++){
14                 for(int k = 0; k < 2; k++){
15                     tmp.mat[i][j] = ((tmp.mat[i][j] +
16                         (mat[i][k] % mod) *
17                         (inp.mat[k][j] % mod)) % mod)
18                     % mod;
19                 }
20             }
21         }
22         return tmp;
23     }
24 };
25 Matrix base;
26 Matrix fast_pow(int exp){
27     if(exp == 1) return base;
28     if(exp % 2 == 0){
29         Matrix res = fast_pow(exp >> 1);
30         return res * res;
31     }
32     Matrix res = fast_pow(exp >> 1);
33     return base * res * res;
34 }
35 int main(){
36     IOS
37     base.mat[0][0] = 1;
38     base.mat[0][1] = 4;
39     base.mat[1][0] = 2;
40     base.mat[1][1] = 3;
41     Matrix output = fast_pow(10);
42     for(int i = 0; i < 2; i++){
43         for(int j = 0; j < 2; j++){
44             cout << output.mat[i][j] << ' ';
45         }
46         cout << '\n';
47     }
48 }

```

5.3 快速計算費氏數列

```

1 #include <bits/stdc++.h>
2 #define IOS
3 ios_base::sync_with_stdio(false);cin.tie(0);cout.tie(0);
4 #define INF 0x3f3f3f3f
5 using namespace std;
6 typedef long long ll;
7 ll mod = 1000000007;

```

```

7 struct Matrix{
8     ll mat[2][2] = {{0}};
9     Matrix operator * (Matrix &inp){
10         Matrix tmp;
11         for(int i = 0; i < 2; i++){
12             for(int j = 0; j < 2; j++){
13                 for(int k = 0; k < 2; k++){
14                     tmp.mat[i][j] = ((tmp.mat[i][j] +
15                                     (mat[i][k] % mod) *
16                                     (inp.mat[k][j] % mod)) % mod)
17                                     % mod;
18                 }
19             }
20         }
21         return tmp;
22     }
23 };
24 Matrix base;
25 Matrix fast_pow(int n){
26     if(n == 1) return base;
27     if(n % 2 == 0){
28         Matrix res = fast_pow(n >> 1);
29         return res * res;
30     }
31     Matrix res = fast_pow(n >> 1);
32     return base * res * res;
33 }
34 int main(){
35     IOS
36     base.mat[0][0] = 1;
37     base.mat[0][1] = 1;
38     base.mat[1][0] = 1;
39     base.mat[1][1] = 0;
40     Matrix output = fast_pow(20);
41     cout << output.mat[0][0] << '\n';
42
43     return 0;
44 }

```

6 常用小扣

6.1 01 背包問題

```

1 // 01 backpack
2 for(int i=1; i<=n; i++){
3     for(int j=x; j>=w[i]; j--){
4         dp[j]=max(dp[j], dp[j-w[i]]+v[i]);
5     }
6 }

```

6.2 Topological

```

1 //拓鋪
2 int N, M, u, v, deg[MAXN]; // deg[i] 紀錄點 i
3 // 被連入邊數
4 vector<int> edge[MAXN];
5
6 /*----- 1. 計算 indegree -----*/
7 cin >> N >> M;
8 while(M--){
9     {
10         cin >> u >> v;
11         edge[u].push_back(v);
12         ++deg[v];
13     }
14 }
15 /*----- 2. 將 indegree 為 0 的點放入 queue 中 -----*/
16 queue<int> q; // 紀錄待拔點
17 for(int i = 0; i < N; ++i)
18     if(deg[i] == 0)

```

```

18         q.push(i);
19
20 /*----- 3. 重複拔點，直到 queue 清空 -----*/
21 while(!q.empty()){
22     {
23         int cur = q.front(); q.pop();
24         cout << cur << "\n";
25         for(int i: edge[cur])
26             {
27                 --deg[i]; // 3-1.
28                 // 相連點 indgree 減一
29                 if(deg[i] == 0) q.push(i); // 3-2. 若該點
30                 // indgree 減至 0，則放入 queue 中
31             }
32     }
33 }

```

6.3 常用

```

1 while(getline(ss, str, 'm')) //以m分割
2     cout << str << endl;
3 int a, b; char c;
4 while(ss >> a >> c >> b) //處理像是 -2/9+9/7
5     cout << a << " " << c << " " << b << endl;
6 //輸出 -2 / 9 endl 9 / 7

```

7 Basic

7.1 GCD

```

1 int gcd(int a, int b){
2     if(b == 0) return a;
3     else return gcd(b, a % b);
4 }

```

7.2 LCM

```

1 int lcm(int a, int b){
2     return a * b / gcd(a, b);
3 }

```

7.3 Leap Year

```

1 bool isLeap(int n){
2     if(n % 100 == 0)
3         if(n % 400 == 0) return true;
4         else return false;
5     if(n % 4 == 0) return true;
6     else return false;
7 }

```

8 Trick

8.1 求根號和解決 pow 精度問題

```

1 // when the number is too large, use powl instead of
2 // pow
3 // will provide you more accuracy.
4 powl(a, b)
5 (int)round(p, (1.0/n)) //nth root of p

```

8.2 查找和二分搜

```
1 // will return address of iterator, call result as
  // iterator;
2 iterator find(iterator first, iterator last, const
  T&value);
3 bool binary_search(iterator first, itee)
```

8.3 $a^b \bmod p$

```
1 // a^b mod p
2
3 long powmod(long base, long exp, long modulus){
4     base %= modulus;
5     long result = 1;
6     while(exp > 0){
7         if(exp & 1) result = (result * base) %
            modulus;
8         base = (base * base) % modulus;
9         exp >> 1;
10    }
11    return result;
12 }
```

8.4 $n! \bmod p$

```
1 // n! mod p
2
3 int factmod(int n, int p){
4     long long res = 1;
5     while(n > 1){
6         res = (res % powmod(p-1, n/p, p)) % p;
7         for(int i=2; i<=n/p; i++)
8             res = (res * i) % p;
9         n /= p;
10    }
11 }
```

8.5 從 1 ~ n 選 M 個數字

```
1 // n >= m choose m number from 1 ~ n
2 void combination(int n, int m){
3     if (n < m) return;
4
5     int a[50] = {0};
6     int k = 0;
7
8     for(int i=1; i<=m; i++) a[i] = i;
9     while(1){
10        for(int i=1; i<=m; i++)
11            cout << a[i] << " ";
12        cout << endl;
13
14        k = m;
15        while(((k>0) && (n-a[k] == m-k)) k --;
16        if(k == 0) break;
17        a[k] ++;
18        for(int i=k+1; i<=m; i++){
19            a[i] = a[i-1] + 1;
20        }
21    }
22 }
```

8.6 二項式係數 Binomial Coefficient (也可以用在 C 幾取幾)

```
1 #define MAXN 100 // largest n or m
2 long binomial_coefficient(n, m) // compute n choose m
3 int n, m;{
4     int i, j;
5     long bc[MAXN][MAXN];
6     for(i = 0; i <= n; i++) bc[i][0] = 1;
7     for(j = 0; j <= n; j++) bc[j][j] = 1;
8     for(i = 1; i <= n; i++)
9         for(j = 1; j < i; j++)
10            bc[i][j] = bc[i-1][j-1] + bc[i-1][j];
11     return bc[n][m];
12 }
```

8.7 LICS

```
1 int a[100] = {0};
2 int b[100] = {0};
3 int f[100] = {0};
4 int n = 0, m = 0;
5 int main(void){
6     cin >> n;
7     for(int i = 1; i <= n; i++) cin >> a[i];
8     cin >> m;
9     for(int i = 1; i <= m; i++) cin >> b[i];
10    for(int i = 1; i <= n; i++){
11        int k = 0;
12        for(int j = 1; j <= m; j++){
13            if(a[i] > b[j] && f[j] > k) k = f[j];
14            else if(a[i] == b[j] && k + 1 > f[j])
15                f[j] = k + 1;
16        }
17    }
```

8.8 m-ary to 10-ary

```
1 string num = "0123456789ABCDE";
2
3 int mToTen(string n, int m){
4     int multi = 1;
5     int result = 0;
6
7     for(int i = n.size() - 1; i >= 0; i--){
8         result += num.find(n[i]) * multi;
9         multi *= m;
10    }
11    return result;
12 }
```

9 Math

9.1 ModInverse

```
1 // 求 a 在模 m 下的模反元素，m 必須是質數
2 ll modInverse(ll a, ll m) {
3     ll res = 1, exp = m - 2;
4     while (exp > 0) {
5         if (exp % 2 == 1) res = (res * a) % m;
6         a = (a * a) % m;
7         exp /= 2;
8     }
9     return res;
10 }
```

9.2 CRT

```

1 ll extendedGCD(ll a, ll b, ll &x, ll &y){
2     if(b == 0){
3         x = 1, y = 0;
4         return a;
5     }
6     ll x1, y1;
7     ll g = extendedGCD(b, a % b, x1, y1);
8     x = y1;
9     y = x1 - (a / b) * y1;
10    return g;
11 }
12
13 ll chineseRemainder(vector<ll> &n, vector<ll> &a){
14     ll prod = 1;
15     for(ll num : n) prod *= num; // Calculate the
        product of all moduli
16
17     ll result = 0;
18     for(int i = 0; i < n.size(); i++){
19         ll p = prod / n[i]; // Product divided by
            current modulus
20         ll x, y;
21         extendedGCD(p, n[i], x, y); // Solve p * x
            1 (mod n[i])
22         result = (result + a[i] * x * p) % prod; //
            Add current term to result
23     }
24     return (result + prod) % prod; // Return the
        result modulo the product
25 }

```

9.3 LinearCongruence

```

1 ll solveLinearCongruence(ll a, ll b, ll m){
2     ll x, y;
3     ll g = extendedGCD(a, m, x, y); // Solve a * x +
        m * y = gcd(a, m)
4     if (b % g != 0) return -1; // No solution if
        gcd(a, m) does not divide b
5
6     x = (x * (b / g)) % m;
7     if (x < 0) x += m; // Ensure the result is
        positive
8     return x;
9 }

```

10 Temporary

10.1 Disjoint Set v1

```

1 int n=1000; //假設共有n個點
2 int dsu[1000];
3 //建一個紀錄集合編號的陣列，若dsu[n]==n，代表為根
4
5 //初始化集合
6 void initt(){
7     for(int i=0;i<=n;i++){
8         //初始每個集合只有自己
9         dsu[i]=i;
10    }
11
12    //查詢
13    //傳入要找尋的目標
14    int findd(int x){
15        if(x!=dsu[x]){
16            //往上尋找祖先，再遞迴更新路上遇到的節點
17            dsu[x]=Find(dsu[x]);
18        }
19        return dsu[x];
20    }

```

```

21 //合併
22 void unionn(int x,int y){
23     int a=findd(x);
24     int b=findd(y);
25     //若集合編號不相同，則進行合併，
26     if(a!=b){
27         dsu[a]=b;
28     }
29 }

```

10.2 Disjoint Set v2

```

1 int Find(int a){
2     return (anc[a] == a ? a : anc[a] = Find(anc[a]));
3 }
4 bool merge(int a, int b){
5     a = Find(a);
6     b = Find(b);
7     if(a == b) return false;
8     if(sz[a] < sz[b]) swap(a, b);
9     anc[b] = a;
10    sz[a] += sz[b];
11    return true;
12 }

```

10.3 Tree DP

```

1 #include <bits/stdc++.h>
2 #define IOS
3     ios_base::sync_with_stdio(false);cin.tie(0);cout.tie(0);
4 #define INF 0x3f3f3f3f
5 using namespace std;
6 typedef long long ll;
7 int dp[205][2];
8 bool uni[205][2];
9 void dfs(vector<int> vec[205], int u){
10     dp[u][0] = 0;
11     dp[u][1] = 1;
12     uni[u][0] = 1;
13     uni[u][1] = 1;
14     for(auto v : vec[u]){
15         dfs(vec, v);
16         dp[u][1] += dp[v][0];
17         dp[u][0] += max(dp[v][0], dp[v][1]);
18         if(dp[v][0] > dp[v][1]){
19             if(!uni[v][0]) uni[u][0] = 0;
20         }
21         else if(dp[v][1] > dp[v][0]){
22             if(!uni[v][1]) uni[u][0] = 0;
23         }
24         if(dp[v][0] == dp[v][1]) uni[u][0] = 0;
25         if(!uni[v][0]) uni[u][1] = 0;
26     }
27 }
28 int main(){
29     IOS
30     int n;
31     while(cin >> n && n){
32         unordered_map<string, int> un;
33         vector<int> vec[205];
34         string boss;
35         cin >> boss;
36         int idx = 0;
37         un[boss] = idx++;
38         memset(uni, 1, sizeof(uni));
39         memset(dp, 0, sizeof(dp));
40         for(int i = 0; i < n - 1; i++){
41             string str1, str2;
42             cin >> str1 >> str2;
43             if(!un[str1]) un[str1] = idx++;
44             if(!un[str2]) un[str2] = idx++;
45             vec[un[str2]].push_back(un[str1]);
46         }
47     }

```

```
46     dfs(vec, un[boss]);
47     cout << max(dp[un[boss]][0], dp[un[boss]][1])
         << ' ';
48     if(dp[un[boss]][0] == dp[un[boss]][1]){
49         cout << "No" << '\n';
50     }
51     else if(dp[un[boss]][0] > dp[un[boss]][1]){
52         cout << (uni[un[boss]][0] ? "Yes" : "No")
             << '\n';
53     }
54     else if(dp[un[boss]][1] > dp[un[boss]][0]){
55         cout << (uni[un[boss]][1] ? "Yes" : "No")
             << '\n';
56     }
57 }
58
59 return 0;
60 }
```